

Workshop 13: SQLite CRUD Hands-on Lab

Node.js, SQLite, Search และ CRUD Operations

เป้าหมาย

Workshop นี้ให้นักศึกษาฝึกใช้งานฐานข้อมูล SQLite ร่วมกับ Node.js โดยเน้น

- การเชื่อมต่อ SQLite
- การสร้าง Table อัตโนมัติ
- INSERT ข้อมูล
- READ และ SEARCH
- UPDATE ข้อมูล
- DELETE ข้อมูล
- การใช้ Parameter Placeholder

Workshop ประกอบด้วย 3 งาน

1. Setup SQLite Database
2. Create, Read and Search Data
3. Update and Delete Data

งานที่ 1 — Setup SQLite Database

สร้าง Node.js Project และเชื่อมต่อกับฐานข้อมูล SQLite

Step 1 — สร้าง Project

เปิด Terminal แล้วพิมพ์

```
mkdir sqlite-workshop  
cd sqlite-workshop  
npm init -y
```

จากนั้นติดตั้ง SQLite Library

```
npm install sqlite3
```

Step 2 — สร้างไฟล์ app.js

สร้างไฟล์

app.js

แล้ว Import Library

```
const sqlite3 =  
  require("sqlite3").verbose();
```

Step 3 — เชื่อมต่อ Database

สร้างฐานข้อมูลชื่อ

workshop.db

ด้วย Code

```
const db =  
  new sqlite3.Database("workshop.db");
```

หากยังไม่มีไฟล์ฐานข้อมูล SQLite จะสร้างไฟล์ขึ้นมาให้อัตโนมัติ

Step 4 — สร้าง Table

สร้าง Table ชื่อ

users

โดยประกอบด้วย

- id
- name
- age

ใช้ Code

```
db.run(`  
  CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT,  
    age INTEGER  
  )  
`);
```

โครงสร้างข้อมูล

หลังจากสร้าง Table แล้ว โครงสร้างข้อมูลจะเป็น

id	name	age
1	Alice	25
2	Bob	30

งานที่ 2 — Create, Read and Search Data

ให้นักศึกษาฝึกเพิ่ม อ่าน และค้นหาข้อมูลจาก SQLite

Step 1 — INSERT ข้อมูล

เพิ่ม User ใหม่

```
db.run(
  "INSERT INTO users (name, age) VALUES (?, ?)",
  ["Alice", 25]
);
```

เพิ่ม User อีกคน

```
db.run(
  "INSERT INTO users (name, age) VALUES (?, ?)",
  ["Bob", 30]
);
```

Parameter Placeholder

ในคำสั่ง

```
VALUES (?, ?)
```

เครื่องหมาย ? ใช้เป็นตำแหน่งสำหรับรับค่าข้อมูล
ค่าจริงจะถูกส่งแยกออกมา เช่น

```
["Alice", 25]
```

วิธีนี้ช่วยให้ Code เป็นระเบียบ และปลอดภัยกว่าการนำค่ามาต่อกับ SQL โดยตรง

Step 2 — อ่านข้อมูล 1 Record

ใช้

```
db.get()
```

ตัวอย่าง

```
db.get(
  "SELECT * FROM users WHERE id = ?",
  [1],
  (err, row) => {
```

```

    if (err) {
      console.log(err);
      return;
    }

    console.log(row);
  }
);

```

`db.get()` เหมาะสำหรับอ่านข้อมูลเพียง 1 Record

Step 3 — อ่านข้อมูลทั้งหมด

ใช้

`db.all()`

ตัวอย่าง

```

db.all(
  "SELECT * FROM users",
  [],
  (err, rows) => {

    if (err) {
      console.log(err);
      return;
    }

    console.log(rows);
  }
);

```

`db.all()` จะคืนค่าข้อมูลหลาย Record ในรูปแบบ Array

Step 4 — Search

ค้นหาผู้ใช้จากชื่อ

```

db.get(
  "SELECT * FROM users WHERE name = ?",
  ["Alice"],
  (err, row) => {

```

```
    if (err) {  
      console.log(err);  
      return;  
    }  
  
    console.log(row);  
  }  
);
```

Challenge

ให้นักศึกษาเพิ่ม User อีกอย่างน้อย 3 คน จากนั้นเขียนคำสั่งเพื่อ

1. แสดง User ทั้งหมด
2. ค้นหา User ด้วย id
3. ค้นหา User ด้วย name

งานที่ 3 — Update and Delete Data

ให้นักศึกษาฝึกแก้ไข และลบข้อมูลใน Database

Step 1 — UPDATE

แก้ไขชื่อของ User ที่มี

id = 1

จาก Alice เป็น Alicia

```
db.run(
  "UPDATE users SET name = ? WHERE id = ?",
  ["Alicia", 1]
);
```

จากนั้นตรวจสอบข้อมูล

```
db.get(
  "SELECT * FROM users WHERE id = ?",
  [1],
  (err, row) => {

    console.log(row);

  }
);
```

Step 2 — UPDATE อายุ

แก้ไขอายุของ User

```
db.run(
  "UPDATE users SET age = ? WHERE id = ?",
  [26, 1]
);
```

Step 3 — DELETE

ลบ User ที่มี

id = 2

ใช้คำสั่ง

```
db.run(  
  "DELETE FROM users WHERE id = ?",  
  [2]  
);
```

จากนั้นดูข้อมูลทั้งหมดอีกครั้ง

```
db.all(  
  "SELECT * FROM users",  
  [],  
  (err, rows) => {  
  
    console.log(rows);  
  
  }  
);
```

ข้อควรระวัง

ก่อนใช้ UPDATE หรือ DELETE ควรระบุเงื่อนไขให้ชัดเจน
ตัวอย่าง

```
DELETE FROM users  
WHERE id = ?
```

ไม่ควรใช้

```
DELETE FROM users
```

หากไม่ได้ต้องการลบข้อมูลทั้งหมดใน Table

คำสั่งสำคัญของ SQLite

คำสั่ง	ใช้ทำอะไร	ผลลัพธ์
<code>new sqlite3.Database()</code>	เชื่อมต่อฐานข้อมูล	Database Connection
<code>db.run()</code>	INSERT / UPDATE / DELETE	เขียนข้อมูล
<code>db.get()</code>	อ่าน 1 Record	Object
<code>db.all()</code>	อ่านหลาย Record	Array

CRUD Flow

การทำงานของ Workshop นี้คือ

Create → INSERT

Read → SELECT

Update → UPDATE

Delete → DELETE

Checklist ก่อนส่งงาน

งานที่ 1 — Database Setup

- ☐ สร้าง Project sqlite-workshop
- ☐ ติดตั้ง sqlite3
- ☐ มีไฟล์ app.js
- ☐ เชื่อมต่อ Database ได้
- ☐ สร้าง Table users
- ☐ ใช้ PRIMARY KEY
- ☐ ใช้ AUTOINCREMENT

งานที่ 2 — Create และ Read

- ☐ ใช้ INSERT เพิ่มข้อมูล
- ☐ ใช้ Parameter Placeholder
- ☐ ใช้ `db.get()`

- ☐ ใช้ `db.all()`
- ☐ ใช้ `SELECT`
- ☐ ใช้ `WHERE` ค้นหาข้อมูล

งานที่ 3 — Update และ Delete

- ☐ ใช้ `UPDATE`
- ☐ ใช้ `DELETE`
- ☐ ระบุ `WHERE id = ?`
- ☐ ตรวจสอบข้อมูลหลัง Update
- ☐ ตรวจสอบข้อมูลหลัง Delete
- ☐ CRUD ทำงานครบทั้ง 4 แบบ

งานที่นักศึกษาต้องส่ง

ให้นักศึกษาส่ง

1. Folder `sqlite-workshop`
2. ไฟล์ `app.js`
3. ไฟล์ Database `workshop.db`
4. Screenshot ผลลัพธ์การ `INSERT`
5. Screenshot ผลลัพธ์การ `SEARCH`
6. Screenshot ผลลัพธ์หลัง `UPDATE` และ `DELETE`

Node.js → SQLite → Create → Read → Update → Delete